



**PROGRAMACION
ESTRUCTURADA
BASICA**

UNIDAD 6. ARCHIVOS BINARIOS

INDICE

1.	QUE ES UN ARCHIVO BINARIO.....	3
2.	COMO SE ACCEDE A UN ARCHIVO BINARIO	3
3.	APERTURA DE UN ARCHIVO BINARIO (FUNCIÓN FOPEN ())	5
4.	COMO CERRAR UN ARCHIVO BINARIO. FUNCIÓN DE CIERRE (FUNCIÓN FCLOSE ())	7
5.	COMO LEER UN REGISTRO DE UN ARCHIVO BINARIO. FUNCIÓN DE ENTRADA (FUNCIÓN FREAD ())	7
6.	COMO ESCRIBIR UN REGISTRO DE UN ARCHIVO BINARIO. FUNCIÓN DE SALIDA (FUNCIÓN FWRITE())	8
7.	COMO LEER UN ARCHIVO BINARIO SIN CONOCER LA CANTIDAD DE REGISTROS. FUNCIÓN FEOF().	9
8.	ARCHIVOS RELACIONADOS.....	10
9.	EJEMPLOS DEL USO DE ARCHIVOS BINARIOS CON ESTRUCTURAS DE DATOS.....	10
10.	CREAR ARCHIVOS BINARIOS DE PRUEBAS CON DATOS PREFIJADOS	13
11.	ACCESO DISCONTINUO A LOS ARCHIVOS BINARIOS.....	15
11.1	1 MODOS DE APERTURA	15
11.2	FUNCIÓN PARA REPOSICIONARSE EN EL ARCHIVO (FSEEK).....	15
11.3	FUNCIÓN PARA CONOCER LA POSICIÓN ACTUAL EN EL ARCHIVO (FTELL)	17
11.4	FUNCIÓN PARA REGRESAR AL COMIENZO DEL ARCHIVO (REWIND)	17
12.	OTRAS FUNCIONES PARA EL MANEJO DE ARCHIVOS BINARIOS	17
13.	ACTUALIZACIÓN DE DATOS SOBRE UN MISMO ARCHIVO BINARIO.....	17
14.	CONOCER LA CANTIDAD DE REGISTROS DE UN ARCHIVO BINARIO	19
15.	BORRADO DE REGISTROS DE UN ARCHIVO BINARIO	20
15.1	BORRADO LÓGICO.....	20
15.2	BORRADO FÍSICO.....	21
15.3	ARCHIVO HISTÓRICO	22
16.	BÚSQUEDA EN ARCHIVOS BINARIOS	22
17.	CORTE DE CONTROL	24
17.1	REGLAS GENERALES PARA APLICAR CORTE DE CONTROL.....	25
17.2	CÓMO APLICAR CORTE DE CONTROL EN DIFERENTES CASOS.....	25
17.2.1	Un Nivel de corte desde un archivo.....	26
17.2.2	Un Nivel de corte desde un archivo generando archivo resumen	28
17.2.3	Un Nivel de corte desde un archivo generando archivos dinámicamente	30
17.2.4	Dos Niveles de corte desde un archivo	32

UNIDAD 6 – ARCHIVOS

OBJETIVOS: Realizar programas que puedan almacenar datos y resultados en una memoria no volátil a fin de respaldar la información generada permitiendo construir programas que no pierdan los datos cargados al finalizar su ejecución.

1. Que es un archivo binario

Los datos con los que se han trabajado hasta el momento han residido en la memoria principal. Sin embargo, las grandes cantidades de datos se almacenan normalmente en un dispositivo de memoria secundaria. Estas colecciones de datos se conocen como archivos (en ingles FILE).

Un archivo binario es una secuencia de bytes que tienen una correspondencia directa con lo almacenado en memoria. Así que no tendrá lugar ninguna traducción de caracteres. Ejemplos de estos archivos son Fotografías, imágenes, texto con formatos, archivos ejecutables (aplicaciones), etc. Los archivos binarios en general se manejan almacenando bloques de información del mismo tipo. Un archivo binario es un conjunto de datos estructurados en una colección de entidades elementales o básicas denominadas registros que son de igual tipo y constan a su vez de diferentes entidades de nivel más bajos denominadas campos. El formato del registro de un archivo binario estará dado por una estructura en el lenguaje C.

En el **Lenguaje C**, un archivo es un concepto lógico que puede aplicarse a muchas cosas desde archivos de disco hasta terminales o una impresora. Se asocia una sentencia con un archivo específico realizando una operación de apertura. Una vez que el archivo está abierto, la información puede ser intercambiada entre este y el programa.

Se puede conseguir la entrada y la salida de datos a un archivo a través del uso de la biblioteca de funciones; el **Lenguaje C** no tiene palabras claves que realicen las operaciones de E/S. La siguiente tabla da un breve resumen de las funciones que se pueden utilizar. Se debe incluir la biblioteca `<stdio.h>`. Observe que la mayoría de las funciones comienzan con la letra "F".

fopen ()	Abre un archivo.
fclose ()	Cierra un archivo.
fread ()	Lee un registro del archivo.
fwrite ()	Guarda un registro en el archivo.
feof()	Devuelve un número distinto de 0 si se llega al final del archivo.

2. Como se accede a un archivo binario

Desde el programa lo que se busca es poder grabar y recuperar información de un dispositivo externo conectado a la computadora que puede ser un disco rígido, un pen drive, una tarjeta de memoria, etc. Es decir, el acceso se hará sobre un componente físico (hardware) que puede tener distinta tecnología (óptica, magnética, etc) y por lo tanto la forma de acceso será diferente.

Como el lenguaje C es de alto nivel no es necesario preocuparse por la forma específica de acceso ya que existe una "abstracción" del lenguaje C que resuelve ese acceso mediante lo que se conoce como flujos o corrientes (stream).

No importa el dispositivo físico donde se encuentre el archivo ya que todos los dispositivos se pueden manejar de la misma forma. Se crea un intermediario entre el programa y el hardware y es finalmente el sistema operativo mediante el uso de drivers (controladores) el que se encarga de acceder a la información a nivel físico.

Para cada archivo que se quiera utilizar se creará un flujo que comunica el programa con el dispositivo externo.

Si existen varias formas para trabajar con archivos la forma de trabajo será mediante el uso de un buffer intermedio. El buffer es un área en la memoria donde se guarda información temporalmente para luego ser llevada al dispositivo externo. Es decir que el programa no graba directamente en el hardware, sino que graba en esa memoria intermedia y luego los datos se toman de esa memoria y a medida que se graban en el dispositivo externo se van borrando de dicha área de memoria.

El buffer se utiliza debido a que las velocidades de acceso son muy diferentes según el hardware conectado. Además, el trabajar directamente en un espacio de memoria es mucho más rápido haciendo que nuestro programa no tenga que esperar los tiempos de acceso y pueda seguir con otros procesos mientras se van grabando los datos físicamente.

Desde el programa se va grabando en el buffer y cuando el buffer se llena o llega a determinada capacidad un proceso se encarga de tomar esos datos y volcarlos al dispositivo físico. Pero este procedimiento es transparente para el programador.

Sin embargo, para que se pueda llevar a cabo dicho procedimiento para cada archivo que se quiera utilizar desde el programa se deben guardar bastantes datos. Para el buffer se debe saber, en que posición de la memoria se encuentra, que tamaño tiene y hasta donde está lleno. Ver Figura 1.

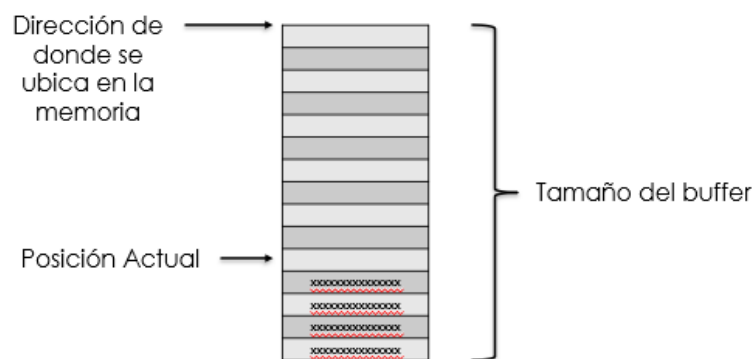


Figura 1. Componentes necesarios para manejar un buffer

Pero además de la información del buffer dicho proceso necesita más datos para poder acceder al archivo. Todos los datos necesarios para el manejo de un archivo están definidos con una estructura definida en la biblioteca `stdio.h` denominada `FILE`.

Esta estructura contiene entre otras cosas (ver figura 2):

- La dirección de memoria del buffer
- El tamaño de buffer
- Hasta donde está lleno el buffer
- El descriptor del archivo (código interno con el cual el sistema operativo identifica los archivos abiertos)
- Indicador de posición dentro del archivo físico (desde que punto se puede comenzar a leer o escribir)
- Una serie de señales, banderas o flags, que dan información del resultado de algunas operaciones.

Definida en stdio.h

```
typedef struct _iobuf
{
    char* _ptr;
    int _cnt;
    char* _base;
    int _flag;
    int _file;
    int _charbuf;
    int _bufsiz;
    char* _tmpfname;
} FILE;
```

Estructura FILE

Tamaño Buffer
Posición Actual
Buffer
Flags
Descriptor del archivo
Indicador de posición en archivo
..
..

El sistema operativo maneja una tabla de Archivos Abiertos

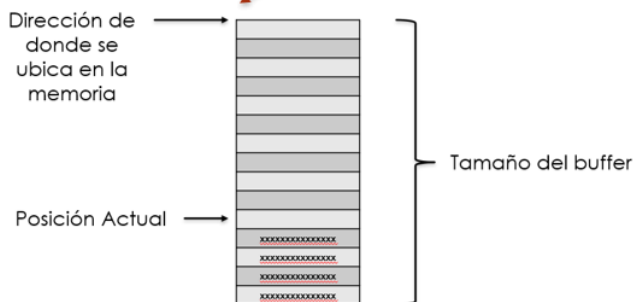


Figura 2: Estructura necesaria para guardar los datos para manejar el flujo de archivos

Todos estos componentes serán administrados de forma transparente al programador ya que solo utilizará algunas funciones que internamente utilizan esta estructura para administrar los archivos.

3. Apertura de un archivo binario (función fopen ())

Antes de utilizar un Archivo en un programa, ya sea para escribir datos en él y/o para leer datos de él, el Archivo debe ser abierto, con lo que se establece un canal de comunicación entre el programa y el Archivo. Y la función para abrir un Archivo se denomina **fopen ()** y su prototipo es:

```
FILE * fopen (char nombreArchivo [ ] , char modoApertura [ ] );
```

El fopen se encargará de crear el flujo, decirle al sistema operativo que abra y prepare para trabajar el archivo y de generar toda la estructura necesaria en la memoria para administrar el buffer y la información del archivo como se vio anteriormente. Dicha función retorna una dirección de memoria de donde se encuentra la estructura para el manejo de archivo.

FILE es el nombre de la estructura definida en stdio.h pero la función retorna un FILE *. El asterisco significa que no retorna una variable del tipo FILE sino que retorna la dirección de memoria donde creó dicha variable. Es decir retorna un puntero a donde se encuentra la variable FILE. En el lenguaje C una variable que guarda una dirección de memoria es un puntero, ya que apunta (o referencia) a otra variable.

Todas las operaciones sobre el Archivo se realizarán a través de ese puntero. Para indicar el Archivo que se desea abrir se usa la cadena de caracteres nombreArchivo la cual puede incluir, además del nombre del Archivo, la ruta completa donde está ubicado, es decir la unidad de almacenamiento y la carpeta. No olvidar que la barra invertida (\) de la ruta debe escribirse dos veces (\\), porque si se escribe sólo una vez es interpretada como secuencia de escape. Si no se coloca la ruta el archivo se busca en la misma carpeta donde se encuentra el ejecutable. Para que el programa pueda ser ejecutado en distintas computadoras sin errores se recomienda no poner una ruta fija.

Con la cadena modoApertura se indicará si se va a leer del archivo, se va a escribir en él o ambas cosas.

Trabajar con archivos secuenciales es decir que solo se puede abrir un archivo para una cosa a la vez y no se puede volver atrás en el archivo. Los modos disponibles para el uso de archivos secuenciales son los siguientes:

- “r”, “rb” se abre para lectura sino existe error
- “w”, “wb” se crea para escritura, si existe lo destruye (ojo no avisa)
- “a”, “ab” modo de escritura donde se abre para agregar a continuación del último dato que está en archivo, sino existe lo crea

La Siguiente tabla resume los modos a utilizar en esta unidad:

Modo	Acción	Archivo ya existe	Archivo no existe	Lectura	Escritura	Posición en el archivo
rb	Lectura Binaria	Correcto	* Error *	Correcto	* Error *	Principio
wb	Escritura Binaria	Borra contenido	Se crea	* Error *	Correcto	Principio
ab	Añadir Binario	Correcto	Se crea	* Error *	Correcto	Final

En la tabla se observa que la apertura de un Archivo con el modo “wb” puede ser peligrosa, ya que si el Archivo existe se perderán todos sus datos. Aunque hay situaciones en las que es esa acción precisamente la que se quiere realizar, destruir todos los datos previos del Archivo. Por otra parte, debe tenerse en cuenta que si se utiliza el modo de apertura “ab” los datos nuevos que se escriban en el Archivo nunca sobrescriben otros datos, sino que siempre se añaden al final.

Si la función fopen () tiene éxito, es decir abre el Archivo sin ningún problema, devolverá el puntero al Archivo. Por el contrario, si se ha producido algún error al intentar la apertura devuelve el valor NULL. Por tanto, después de ejecutar la función fopen (), el programa deberá comprobar siempre si el puntero devuelto vale NULL.

Ejemplos:

```

/* Especificando solo el nombre del archivo lo busca en la misma carpeta que el
ejecutable */
FILE *fp;
fp = fopen("C:\\archivos\\ventas.dat", "rb");
if(fp == NULL)
{
    printf ("ERROR al abrir el archivo de ventas");
    getch();
    exit(1);
}

//Especificando unidad, carpeta y nombre de Archivo
FILE *fp;
fp = fopen("C:\\archivos\\ventas.dat", "rb");
  
```

```
if(fp == NULL)
{
    printf ("ERROR al abrir el archivo de ventas");
    getch();
    exit(1);
}
```

4. Como Cerrar un archivo binario. Función de Cierre (función **fclose ()**)

Cuando un programa deja de necesitar un Archivo, éste debe cerrarse, cortándose por tanto el canal de comunicación entre el programa y el Archivo. Para cerrar un Archivo se utiliza la función **fclose ()**.

La función **fclose ()** cierra un Archivo que fue abierto mediante una llamada a **fopen ()**. Escribe toda la información que todavía se encuentre en el buffer en el disco y realiza un cierre formal del archivo a nivel del sistema operativo. Libera toda la estructura creada en memoria para el manejo del flujo del archivo. Un error en el cierre de un Archivo puede generar todo tipo de problemas, incluyendo la pérdida de datos, destrucción de archivos y posibles errores intermitentes en el programa, cuya sintaxis es:

```
fclose (pf);
```

Donde “pf” es el puntero al archivo devuelto por la llamada a **fopen ()**. Generalmente, esta función solo falla cuando un disco se ha retirado antes de tiempo o cuando no queda espacio libre en el mismo.

5. Como leer un registro de un archivo binario. Función de Entrada (función **fread ()**)

La función **fread ()** trabaja con registros de longitud constante. Es capaz de leer desde un Archivo, uno o varios registros de la misma longitud y a partir de una dirección de memoria determinada. Se debe asegurar de que haya espacio suficiente para contener la información leída. La sintaxis de esta función es:

En lenguaje C:

```
fread( <Dir_Variable> , < n°_bytes>, <cantidadDeRegistros>, <variable_file>);
```

La función **fread ()** significa que se leerá del Archivo que apunta la <variable_file> el número de bytes <n°_bytes> tantas veces como indique <cantidadDeRegistros >, dejando dichos bytes grabados en memoria a partir de la dirección <Dir_Variable>. Esta dirección será la de una variable con el tamaño suficiente para guardar los datos leídos. El número de bytes que se leen será por tanto el resultado del producto:

<n°_bytes> * < cantidadDeRegistros >

Ejemplos:

```
int num;
fread(&num, sizeof(int), 1, fp);
//Se leen 4*1=4 bytes del archivo, que caben en num.
float Notas[5];
fread(Notas, sizeof(float), 5, fp);
//Se leen 4*5=20 bytes del fichero, que caben en Notas
```

En el Ejemplo N°1, en el segundo caso se leerán 20 bytes, ya que se leerán 5 veces el número de bytes indicado por **sizeof(float)**, o sea $4 \times 5 = 20$. Por tanto, se leen 5 números float desde el archivo y se guardan en el array Notas, cuyo tamaño es de 20 bytes.

6. Como escribir un registro de un archivo binario. Función de Salida (función `fwrite()`)

La función **`fwrite()`** permite escribir datos en un archivo como una sucesión de bytes. La sintaxis de esta función es:

```
fwrite( <Dir_Variable> , < n°_bytes>, <cantidadDeRegistros>, <variable_file>);
```

A partir de la dirección de memoria <Dir_Variable> se leerán tantos bytes como se indique en <n°_bytes> tantas veces como se especifique en <cantidadDeRegistros> y se escribirán en el Archivo que apunta la < variable_file >. El número de bytes escritos será por tanto el resultado del producto: <n°_bytes> * <cantidadDeRegistros>.

Ejemplos:

```
int num=1067;
fwrite(&num, sizeof(int), 1, fp);
//Graba 4 bytes en el archivo, desde la dirección num, por tanto el valor 1067
queda escrito en el archivo.
float Notas[5]={5.5, 7.25, 8.5, 4.75, 9.5};
fwrite(Notas, sizeof(float), 5, fp);
//Se escriben 4*5=20 bytes en el archivo, desde la dirección Notas, quedando las
5 notas grabadas en el archivo.
```

La función **`fwrite()`** devuelve el número de veces que ha escrito el <n°_bytes>, que no siempre coincide con <cantidadDeRegistros>, ya que puede producirse algún error. Por tanto, si el valor que devuelve **`fwrite()`** no coincide con <cantidadDeRegistros> es que ha ocurrido un error.

Ejemplo . Escribir un número real en un Archivo. Leerlo en otra variable real.

```
FILE *fp;
float var;
float num = 12.23;
if ((fp = fopen("prueba.dat", "wb")) == NULL )
{
    printf("No se puede abrir.\n");
    getch(); exit(1);
}
fwrite( &num, sizeof(float), 1, fp);
fclose(fp);
if ((fp = fopen("prueba.dat", "rb")) == NULL )
{
    printf("No se puede abrir.\n");
    getch(); exit(1);
}
fread (&var, sizeof(float), 1, fp );
fclose(fp);
```

Uno de los usos más comunes de **`fread()`** y **`fwrite()`** es el manejo de Archivo en forma de conjunto de registros, donde cada registro está dividido en campos. Para ello en el programa debe definirse un tipo de estructura de datos con el mismo formato que el registro del Archivo, con el objeto de leer y escribir registros completos con las funciones **`fread()`** y **`fwrite()`** respectivamente, en lugar de leer y escribir campos concretos.

Ejemplo. Teniendo un vector de estructuras ya cargado con datos, deberá grabarse en un Archivo y después dejarlo en otro vector de estructuras.

```
struct datos
{
    char nombre[20];
    char apellido[40];
};
```



```
int main ()
{
    FILE *fp;
    int i;
    struct datos lista1[30], lista2[30];
    //Suponemos que lista1 ya contiene datos.

    if ( (fp = fopen("fich.dat", "wb")) == NULL )
    {
        printf("No se puede abrir.\n");
        getch(); exit(1);
    }
    //Escribe 30 registros desde el array lista1.
    for ( i = 0; i < 30; i++)
        fwrite(&lista1[i], sizeof(struct datos), 1, fp)
    fclose(fp);
    if ( (fp = fopen("fich.dat", "rb")) == NULL )
    {
        printf("No se puede abrir.\n");
        getch(); exit(1);
    }
    //Lee 30 registros, dejándolos en el array lista2.
    for ( i = 0; i < 30; i++)
        fread(&lista2[i], sizeof(struct datos), 1, fp)
    fclose(fp);
    return 0;
}
```

7. Como leer un archivo binario sin conocer la cantidad de registros. Función **feof()**.

Cada vez que se lee de un archivo con la función **fread**, los datos son transferidos a la memoria y el indicador de posición del archivo queda listo para la lectura del siguiente registro justo a continuación del último dato leído. Si se intenta leer y se encuentra que ya no hay datos en el archivo, entonces dentro de la estructura **FILE** se activa una bandera (flag) que indica que se llegó al final del archivo y por lo tanto no se leyeron datos. Para consultar el estado de dicha bandera se utiliza la siguiente función:

```
int feof(FILE *);
```

Su prototipo se encuentra en **<stdio.h>**. Devuelve un número distinto de 0 si se ha alcanzado el final del archivo, si aún hay datos devuelve un 0.

Cuando no se conoce la cantidad de registros que posee un Archivo, se puede utilizar la función **feof()**, la cual permitirá leer la información hasta el fin de Archivo.

Esta función debe utilizarse **luego** de hacer una lectura sobre el archivo con **fread**.

Ejemplo:

```
struct datos
{
    char nombre[20];
    char apellido[40];
};

int main ( )
{
    FILE *fp;
    int i;
```

```
struct datos lista[30];

// Se supone que el archivo no va a contener más de 30 registros.

if ( (fp = fopen("fich.dat", "rb")) == NULL )
{
    printf("No se puede abrir.\n");
    getch(); exit(1);
}
//Lee los registros del archivo hasta el final del mismo.
i = 0;
fread(&lista[i], sizeof(struct datos),1,fp)
while ( !feof (fp) )
{
    i++;
    fread(&lista[i], sizeof(struct datos),1,fp)
}
fclose(fp);
return 0;
}
```

8. Archivos relacionados

Al trabajar con archivos muchas veces la información esta separada en distintos archivos y relacionadas por algún campo clave, por ejemplo, se puede tener un archivo con productos que tenga código, precio y descripción de cada producto, en ese caso el código es un campo clave que identifica unívocamente a cada producto. Por otro lado, si se registran las ventas de la empresa cuando se refiera a que producto se vendió el archivo de ventas solamente va a contener el código de producto vendido ya que el resto de la información como el precio y la descripción se recupera del archivo relacionado. Al trabajar con archivos relacionados en general se supone que el archivo de ventas no va a contener códigos de productos que no existan ya que la integridad de los datos fue chequeada al momento de creación de los archivos. Sin embargo, como cada archivo se puede manejar por separado en algún caso puede ocurrir que se borre algún registro y se pierda la relación haciendo imposible recuperar los datos. Estos problemas en sistemas comerciales se solucionan usando entornos que manejan los datos de forma integral (sistemas gestores de base de datos) y hacen que no se permita borrar un registro si se está utilizando en algún lado relacionado.

En esta materia al trabajar con archivos separados, si el planteo del problema a resolver no lo aclara se supone que el dato relacionado siempre se va a encontrar. En otros problemas planteados se pide chequear la integridad mostrando algún mensaje de error por pantalla o grabando un archivo con los registros no encontrados.

9. Ejemplos del uso de archivos binarios con estructuras de datos

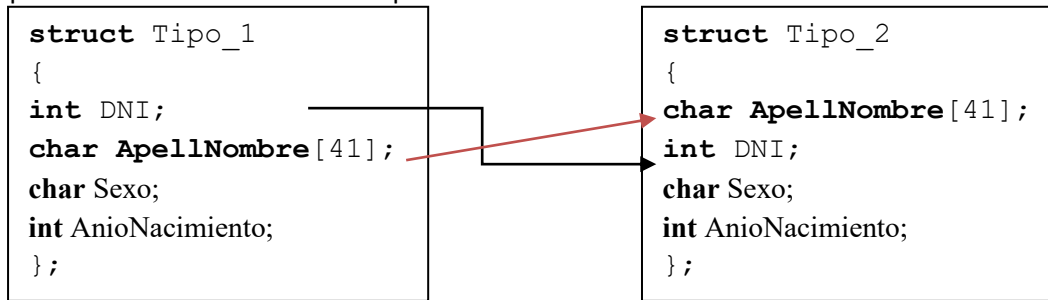
El uso de estructuras de datos en el manejo de “archivos binarios”, resulta imprescindible pues la esencia de estos archivos binarios es la grabación y recuperación de registros es decir estructuras de datos (**struct**).

Tanto para la recuperación (lectura) o grabación (escritura) de un registro del archivo el mismo se realiza a través de una variable tipo estructura (**struct**) que obedezca al diseño del registro del archivo.

Es importante tener en cuenta que para poder recuperar (lectura) o grabar (escritura) la información que se encuentra en un archivo se debe conocer exactamente como está formada la estructura, porque será la única manera que se pueda trabajar con archivos. En el ejemplo que se desarrolla a continuación se corresponden a dos estructuras distintas, aunque ambas posean los mismos datos.

La estructura “**struct** Tipo_1” y la estructura “**struct** Tipo_2” contienen los mismos datos y la misma cantidad de bytes (50 bytes), pero son al mismo momento diferentes porque la información dentro de ambas se encuentra en

órdenes diferentes. Para que dos estructuras sean exactamente iguales deben coincidir totalmente y no es el caso anterior por lo cual son distintas e incompatibles en su tratamiento.



Ejemplo 1:

Es un programa que permite grabar los datos de 10 alumnos en un archivo llamado "AlumnosUNLaM.dat".

```

#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
struct FECHA
{
    int Dia;
    int Mes;
    int Anio;
};
struct PERSONA
{
    long int DNI;
    char ApellNombre[41];
    char Sexo;
    struct FECHA Nacimiento;
};
int main()
{
    struct PERSONA alumno;
    int i;
    FILE *pf;
    pf = fopen("AlumnosUNLaM.dat" , "wb");
    if (pf==NULL)
    {
        printf("No se pudo crear el archivo.");
        getch();
        exit (1);
    }
    for(i=0 ; i < 10; i++)
    {
        printf("Ingrese Numero de DNI");
        scanf("%d", &alumno.DNI);
        printf("Ingrese Numero de Apellido y nombre");
        fflush(stdin);
        gets(alumno.ApellNombre);
        printf("Ingrese Sexo");
        fflush(stdin);
        scanf("%c", &alumno.Sexo);
        printf("Ingrese Numero del Dia de Nacimiento");
        scanf("%d", &alumno.Nacimiento.Dia);
        printf("Ingrese Numero del Mes de Nacimiento");
        scanf("%d", &alumno.Nacimiento.Mes);
        printf("Ingrese Numero del Año de Nacimiento");
        scanf("%d", &alumno.Nacimiento.Anio);
        fwrite(&alumno, sizeof(struct PERSONA), 1, pf);
        /*Aquí la función que permite graba un registro en un archivo binario. Se indica el nombre
        de la variable estructura " alumno " (registro) que se va a grabar pf que es el puntero
        del archivo a grabar. La funcion sizeof determina el largo del registro por eso esta
        invocada la estructura PERSONA (58 bytes). */
    }
    return 0;
}
    
```

Ejemplo 2:

Es un programa que permite grabar los datos que se encuentra en un array (vector) de estructura de datos (los cuales fueron ingresados por teclado dentro de una función llamada "CargaDatosAlumnos"), en un archivo llamado "DatosAlumnosUNLaM.dat"

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
struct FECHA
{
    int Dia;
    int Mes;
    int Anio;
};
struct PERSONA
{
    long int DNI;
    char ApellNombre[41];
    char Sexo;
    struct FECHA Nacimiento;
} ;
void CargaDatosAlumnos (struct PERSONA []); //Prototipo de la función
int main()
{
    struct PERSONA Alumnos[50];
    int i;
    FILE *pf;

    CargaDatosAlumnos( Alumnos );// Llamada de la función
    pf = fopen("DatosAlumnosUNLaM.dat" , "wb");
    if (pf==NULL)
    {
        printf("No se pudo crear el archivo.");
        getch();
        exit (1);
    }
    for(i=0 ; i < 50; i++)
    {
        fwrite(&Alumnos[i], sizeof(struct PERSONA), 1, pf);
        //Aquí la función que permite graba un registro a la vez en un archivo binario.
    }
    fclose(pf);
    return 0;
}

void CargaDatosAlumnos (struct PERSONA Alum[]) //Declaración de la función
{
    int i;
    for(i=0 ;i < 50; i++)
    {
        printf("Ingrese Numero de DNI");
        scanf("%d", & Alum[i].DNI);
        printf("Ingrese Apellido y nombre");
        fflush(stdin);
        gets(Alum[i].ApellNombre);
        printf("Ingrese Sexo");
        fflush(stdin);
        scanf("%c", & Alum[i].Sexo);
        printf("Ingrese Numero del Dia de Nacimiento");
        scanf("%d", & Alum[i].Nacimiento.Dia);
        printf("Ingrese Numero del Mes de Nacimiento");
        scanf("%d", & Alum[i].Nacimiento.Mes);
        printf("Ingrese Numero del Año de Nacimiento");
        scanf("%d", & Alum[i].Nacimiento.Anio);
    }
}
```

Ejemplo 3 :

Es un programa que recupera (lee) los datos que se encuentran en un archivo llamado "DatosAlumnosUNLaM.dat", y luego guardarlos en un arreglo (vector) de estructura de datos, los cuales se muestran por pantalla por medio de una función llamada "DatosAlumnos". Para realizar este ejemplo se debe compilar siempre y cuando exista el archivo llamado "DatosAlumnosUNLaM.dat" y que contenga 50 registros.

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

struct FECHA
{
    int Dia;
    int Mes;
    int Anio;
};
struct PERSONA
{
    long int DNI;
    char ApellNombre[41];
    char Sexo;
    struct FECHA Nacimiento;
};
void DatosAlumnos (struct PERSONA []); //Prototipo de la función
int main()
{
    struct PERSONA Alumnos[50];
    int i;
    FILE *pf;
    pf = fopen("DatosAlumnosUNLaM.dat", "rb");
    if (pf==NULL)
    {
        printf("No se pudo abrir el archivo.");
        getch();
        exit (1);
    }
    for(i=0 ; i < 50; i++)
    {
        fread(&Alumnos[i], sizeof(struct PERSONA), 1, pf);
        //Aquí la función que permite recuperar (leer) un registro a la vez en un archivo binario.
    }
    DatosAlumnos ( Alumnos ); // Llamada de la función
    fclose(pf);
    return 0;
}

/*Declaración de la función. Permite recibir como parámetro un arreglo (array) de estructura de
datos y luego mostrarlos por pantalla.*/
void DatosAlumnos (struct PERSONA Alum[])
{
    int i;
    for(i=0 ; i < 50; i++)
    {
        printf("\nFecha de Nacimiento: %d-%d-%d Nro DNI: %d nombre: %s sexo %c",
            Alum[i].Nacimiento.Dia, Alum[i].Nacimiento.Mes, Alum[i].Nacimiento.Anio, Alum[i].DNI,
            Alum[i].ApellNombre, Alum[i].Sexo);
    }
}
```

10. Crear archivos binarios de pruebas con datos prefijados

Muchas veces se desarrolla un algoritmo considerando que ya existe determinado archivo con datos previamente cargados. Pero sino se dispone de un archivo de pruebas no se puede comprobar su correcto funcionamiento.

Además, es bueno siempre tener distintos conjuntos de datos de prueba para corroborar el correcto funcionamiento de un programa abarcando distintas alternativas.

Para ello se deben crear en forma manual los archivos de prueba. Una forma de crearlos es permitiendo el ingreso de datos por teclado y grabando los archivos, pero este proceso requiere de validaciones y carga de datos por teclado lo que puede llevar a errores y bastante tiempo de desarrollo. Un método más efectivo para las pruebas es definir el conjunto de datos que se requiere que tenga el archivo y armar un programa que genere el archivo con esos datos prefijados. A continuación, se muestra un código de ejemplo para generar un archivo de prueba de productos, pero el código está diseñado para que rápidamente pueda modificarse para generar cualquier otro archivo binario.

El nombre del archivo a generar se declara como una constante con `define` y luego se declara la estructura que define el formato del registro del archivo:

```
#include <stdio.h>
#define ARCHIVO "productos.dat"

typedef struct
{
    int codigo;
    char descripcion[31];
    float precioUnitario;
    char tipo;
} T_Registro;
```

En el `main` se declara un vector de estructuras del tipo del formato del registro y se lo inicializa con los valores preestablecidos deseados. Según sea el formato del registro del archivo a generar serán los datos y el orden de los campos que debe establecerse para armar los registros de prueba. El siguiente ejemplo inicializa un vector con 6 registros de productos, donde entre llaves se especifican los campos de cada registro en el orden declarado en la estructura, en este caso Código, Descripción, Precio y Tipo. Pueden definirse todos los registros que se deseen ya que el tamaño del vector será calculado automáticamente ya que se realiza en la inicialización del mismo.

```
int main()
{
    int i, cant;
    T_Registro vec[]={
        {100, "PINZA CURVA", 3250.38, 'I'},
        {111, "PINZA RECTA", 1250.50, 'N'},
        {200, "CABLE X100", 5000.00, 'N'},
        {210, "CABLE COBRE X50", 9023.00, 'I'},
        {915, "TUERCA 24", 123.00, 'N'},
        {999, "SELLADOR", 1020.76, 'I'}
    };
}
```

Luego se procede a abrir el archivo en modo escritura y se graban los registros del vector en el archivo. El cálculo de la cantidad de registros se realiza en forma automática para no tener que modificar nada adicional en el código.

```
FILE *fp;
cant = sizeof(vec) / sizeof(T_Registro);
fp = fopen(ARCHIVO, "wb");
for (i=0; i<cant; i++)
    fwrite(&vec[i], sizeof(T_Registro), 1, fp);
fclose(fp);

return 0;
}
```

De esta forma con este simple programa se pueden generar los archivos de prueba deseados, simplemente se deben definir los campos del formato del registro y establecer los valores deseados en la inicialización del vector de estructuras, además de definir el nombre del archivo a generar en la constante.

11. Acceso discontinuo a los archivos binarios

Con las funciones y los modos de apertura vistos y en la unidad 3 solo era posible acceder a los archivos de forma secuencial, es decir que de forma automática ya sea al leer o al escribir en un archivo el indicador de posición en el archivo siempre se mueve hacia adelante y no se puede volver a un registro que ya fue leído. Tampoco era posible abrir un archivo en modo mixto de lectura/escritura obligando a tener que cerrar el archivo para abrirlo en otro modo.

En esta unidad se utilizarán modos de apertura y funciones para poder acceder de forma no secuencial a los archivos.

11.1 Modos de Apertura

Como se trabajará con archivos binarios los modos de aperturas que se utilizarán serán los de la tabla 1.

Tabla 1: Modos de apertura de archivos binario para acceso no secuencial

Modo	Función	Ubicación inicial en el archivo
r+b	Lectura y escritura, el archivo debe existir	Al comienzo
w+b	Escritura y lectura, crea el archivo y si ya existe lo sobrescribe	Al comienzo
a+b	Agregar y lectura, crea el archivo y si existe no lo sobrescribe ya se posiciona al final para agregar registros. IMPORTANTE: al abrir un archivo en este modo siempre la escritura se hará al final del archivo sin importar si se cambia el indicador de posición en el archivo utilizando fseek que se explica a continuación	Al comienzo, pero siempre al escribir escribe al final

11.2 Función para reposicionarse en el archivo (fseek)

Las funciones fread y fwrite utilizadas para leer y escribir del archivo hacen que el indicador de posición en el archivo se mueva hacia al final del archivo luego de leer o escribir avanzando de forma automática. Esto es útil ya que una

vez que leemos la siguiente lectura nos dará el siguiente registro sin necesidad de realizar ninguna operación adicional. De igual forma al escribir el indicador de posición quedará luego del registro que se escribe haciendo que si se vuelve a escribir no se pierda la información. Estos modos secuenciales son útiles para realizar una sola operación en simultaneo sobre un archivo, es decir se abre para lectura y siempre se lee o se abre para escritura y siempre se escribe.

Pero ahora con los modos mixtos es posible ir leyendo registros y en algún momento querer escribir por ejemplo para modificar un dato del registro leído y por lo tanto tendremos que poder regresar el indicador de posición en archivo al registro que acabamos de leer recordando que automáticamente este indicador se movió al registro siguiente.

La función para posicionarse en el archivo es `fseek` disponible al igual que todas las funciones de archivos en la biblioteca `stdio.h`. El formato de la función es el siguiente:

`fseek (FILE *fp, long int despBytes, int referencia )`

donde

`fp` es el puntero al archivo sobre el cual se desea desplazarse

`despBytes` es la cantidad de bytes que se desea desplazarse en el archivo

`referencia` es desde que ubicación se va a realizar dicho desplazamiento pudiendo tomar los valores mostrados en la tabla 2.

Tabla2: valores posibles para la referencia desde donde se realizará el desplazamiento con el `fseek`

valor	Función	Constante que puede utilizarse en lugar del número
0	Se desplaza desde el comienzo del archivo. Siempre el desplazamiento deberá ser un valor positivo en este caso	SEEK_SET
1	Se desplaza desde la posición actual. El desplazamiento podrá ser tanto positivo como negativo según la ubicación actual y hacia donde se necesita desplazarse	SEEK_CUR
2	Se desplaza desde el final del archivo. Siempre el desplazamiento deberá ser un valor negativo en este caso	SEEK_END

De esta forma es posible realizar tanto lecturas como escrituras sobre un archivo abierto, pero antes de cambiar de modo es necesario reposicionarse en el archivo utilizando `fseek` o liberar el buffer del flujo con `fflush`. Es decir inmediatamente luego de un `fread` no es posible tener un `fwrite` sin antes realizar un `fseek` de reposicionamiento (incluso si el desplazamiento es de 0 bytes es necesario realizarlo ya que se limpian flags internos de la estructura `FILE`). De igual forma luego de un `fwrite` no será posible realizar un `fread` sin antes realizar un `fseek` o realizar un `fflush` que envíe los datos a escribir del buffer al archivo físico.

11.3 Función para conocer la posición actual en el archivo (ftell)

En ocasiones es necesario conocer la posición actual dentro del archivo y para ello es posible utilizar la función ftell que tiene el siguiente formato

```
long ftell (FILE *)
```

La función ftell recibe un puntero del archivo y retorna un entero indicando la cantidad de bytes desde el origen del archivo (long indica un entero largo pero se recuerda que en el entorno actual el entero largo y el entero son ambos de 4 bytes).

Con el dato de la cantidad de bytes de la posición actual en el archivo es posible calcular el número de registro en el cual se encuentra posicionado dividiendo por el tamaño de cada registro.

Registro actual = ftell(FILE *) / sizeof(estructura que define el tamaño del registro)

11.4 Función para regresar al comienzo del archivo (rewind)

```
void rewind (FILE * FP);
```

Esta función hace que el indicador de posición en el archivo vuelva al inicio y resetee todos los flags de la estructura FILE. Por el ejemplo el flag de fin de archivo, el de error, de modo de acceso, etc. También puede realizarse esta misma acción utilizando el fseek desplazándose 0 bytes desde el inicio.

12. Otras funciones para el manejo de archivos binarios

Dentro de la biblioteca stdio.h existen otras funciones que serán de utilidad para el manejo de archivos. Las que se utilizaran están resumidas en la tabla 3

Tabla 3: Funciones adicionales para el manejo de archivos

Función	Utilidad	Uso
int remove (char [])	Eliminar un archivo	Esta función recibe un string correspondiente al nombre físico del archivo que se desea eliminar. Retorna 0 si se eliminó correctamente.
int rename (char nombreViejo[], char nombreNuevo[])	Cambiar de nombre un archivo	Esta función recibe dos strings el primero con el nombre actual de un archivo y el segundo el nombre por el cual se desea cambiarlo. Retorna 0 si se pudo realizar el cambio de nombre correctamente.

13. Actualización de datos sobre un mismo archivo binario

Con las funciones de reposicionamiento y los modos de apertura mixtos es posible modificar uno o más registros directamente en el archivo Para ver la actualización se plantea el siguiente caso de aplicación:

Se dispone de un archivo con precios de productos con el siguiente formato de registro:

- Código de Producto (entero)
- Descripción (texto de 30 caracteres máximo)
- Precio Unitario (float)
- Tipo de Producto (char P de fabricación propia, I importado, N nacional)

Debido a un cambio en la normativa impositiva vigente a todos los productos importados se les agrega un costo adicional del 20%.

El procedimiento para resolver este requerimiento será el siguiente:

- Se abre el archivo en modo r+b, es decir en modo lectura para que comience del principio pero que también deje escribir en cualquier posición del archivo.
- Por cada registro que se lee se chequea si es importado.
- Si es importado entonces se incrementa el precio en la variable del tipo estructura en memoria donde se leyó el registro y luego se debe actualizar en el archivo. Para ello se debe mover el indicador de posición del archivo hacia atrás un registro utilizando fseek. Luego se debe escribir el registro
- Se continua con el proceso de lectura chequeando todos los registros hasta el fin del archivo y repitiendo el procedimiento pero se debe recordar que entre cambios de modo lectura-escritura o escritura-lectura se debe realizar un reposicionamiento en el archivo con fseek o un fflush en el caso del paso de escritura a lectura.

A continuación, se muestra el código fuente del proceso descrito:

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
typedef struct
{
    int codigo;
    char descripcion[31];
    float precioUnitario;
    char tipo;
} SProducto;

void MostrarProductos();
void IncrementarPreciosPorTipo(char, int);
int main()
{
    MostrarProductos();
    IncrementarPreciosPorTipo('I', 20);

    printf("\n\n---Precios Actualizados---\n");
    MostrarProductos();

    return 0;
}
```

```

void IncrementarPreciosPorTipo(char tipo, int porInc)
{
    FILE * ap;
    SProducto producto;
    ap = fopen("productos.dat", "r+b");
    if (ap==NULL)
    {
        printf("Error al abrir el archivo de productos.");
        getch();
        exit(1);
    }
    fread(&producto, sizeof(producto), 1, ap);
    while (!feof(ap))
    {
        if (toupper(producto.tipo)==toupper(tipo))
        {
            producto.precioUnitario += (producto.precioUnitario * porInc) / 100;
            fseek(ap, sizeof(SProducto) * -1, SEEK_CUR);
            fwrite(&producto, sizeof(producto), 1, ap);
            fflush(ap);
            //fseek(ap, 0, SEEK_CUR); //para volver al modo neutro
        }
        fread(&producto, sizeof(producto), 1, ap);
    }
    fclose(ap);
}

void MostrarProductos()
{
    FILE * ap;
    SProducto producto;
    ap = fopen("productos.dat", "rb");
    if (ap==NULL)
    {
        printf("Error al abrir el archivo de productos.");
        getch();
        exit(1);
    }
    printf("%6s %-30s %8s %4s", "CODIGO", "DESCRIPCION", "PRECIO", "TIPO");
    fread(&producto, sizeof(producto), 1, ap);
    while (!feof(ap))
    {
        printf("\n%6d %-30s %8.2f %4c", producto.codigo, producto.descripcion, \
            producto.precioUnitario, producto.tipo);
        fread(&producto, sizeof(producto), 1, ap);
    }
    fclose(ap);
}

```

14. Conocer la cantidad de registros de un archivo binario

Para conocer la cantidad de registros de un archivo es posible utilizar la función ftell vista anteriormente que retorna la posición actual dentro del archivo. El procedimiento es el siguiente:

1. Se abre el archivo en modo lectura
2. Se mueve el indicador de posición de archivo al final utilizando fseek
3. Se invoca a la función ftell para obtener el desplazamiento en bytes desde el inicio de archivo y se divide esa cantidad de bytes por el tamaño de la estructura del formato de registro del archivo,

El siguiente código de ejemplo muestra una función de cómo obtener el tamaño en bytes de un archivo y la cantidad de registros considerando que el archivo contiene registros grabados con una estructura llamada SProducto

```
void ContarRegistros()
{
    FILE * ap;
    int bytes, registros;
    ap = fopen("productos.dat", "r+b");
    if (ap==NULL)
    {
        printf("Error al abrir el archivo de productos.");
        getch();
        exit(1);
    }
    fseek(ap, 0, SEEK_END);
    bytes = ftell(ap);
    registros = bytes / sizeof(SProducto);
    printf("\nCantidad de bytes del archivo: %d", bytes);
    printf("\nCantidad de registros del archivo: %d", registros);
    fclose(ap);
}
```

15. Borrado de registros de un archivo binario

Debido a que un archivo es una sucesión de bytes no es posible simplemente eliminar un registro de un archivo. Por lo tanto, para poder borrar registros hay dos formas de trabajo el borrado físico pasando por un archivo intermedio o el borrado lógico.

15.1 Borrado Lógico

El borrado lógico implica agregar a cada registro un campo que actúe como indicador de si el registro está vigente o no. Por ejemplo dada la siguiente estructura de productos:

```
typedef struct
{
    int codigo;
    char descripcion[31];
    float precioUnitario;
    char tipo;
} SProducto;
```

Para poder realizar un borrado lógico se debe agregar un campo adicional que indique si ese registro fue eliminado o no. Ese campo puede ser un entero S o N o un número 0 o 1 por ejemplo. Es una definición que debe tomar el programador a la hora de diseñar sus registros. Por ejemplo, la estructura podría definirse de la siguiente forma:

```
typedef struct
{
    char eliminado;
    int codigo;
    char descripcion[31];
    float precioUnitario;
    char tipo;
} SProducto;
```

Donde se agrega una variable del tipo char para indicar si el registro fue eliminado o no. En este caso se usa una variable char ya que es la variable que ocupa la menor cantidad de espacio (1 byte) entonces podría guardarse por defecto siempre la letra N indicando que no fue eliminado y cuando se desea eliminar cambiar esa N por una S. También puede guardarse un número 0 o 1 recordando que internamente en realidad las variables char guardan números que luego si se muestran con el formato %c son interpretados de acuerdo con el código ASCII.

En cualquier caso, esta forma de trabajo trae varios problemas:

- Se agrega un campo a cada registro lo que ocupa lugar haciendo que el archivo sea más pesado
- Se deben modificar todas las funciones de lectura y trabajo sobre el archivo para que no tome en cuenta aquellos archivos marcados como eliminados. Lo que hace más lento el proceso ya que se recorren registros que no se utilizan.
- Si la eliminación es frecuente el archivo va a seguir creciendo en tamaño total en bytes con datos que no se utilizan y por lo tanto es recomendable realizar algún proceso que pueda ejecutarse periódicamente que borre físicamente los registros marcados como eliminados para achicar el archivo y dejarlo solo con datos útiles.

15.2 Borrado Físico

Para borrar físicamente un registro de un archivo la única opción es crear un archivo temporal donde simplemente no se copien los registros que se desean eliminar y luego con ese archivo temporal sobre escribir el archivo original.

A continuación, se muestra un ejemplo donde dado un archivo se desea eliminar el registro de un producto dado su código.

El formato de registro del archivo es el siguiente:

```
typedef struct
{
    int codigo;
    char descripcion[31];
    float precioUnitario;
    char tipo;
} SProducto;
```

Y el archivo que contiene los productos se llama productos.dat. A continuación, se muestra el código de la función que realiza el borrado físico copiando todos los registros de un archivo a otro excepto el del código buscado y luego sobrescribe el archivo original.

```
void EliminarProducto(int codigoABorrar)
{
    FILE * archOrig, *archTemp;
    SProducto reg;

    archOrig = fopen("productos.dat", "rb");
    archTemp = fopen("productos.tmp", "wb");

    if (archOrig==NULL || archTemp==NULL)
    {
        printf("Error al abrir el archivo de productos.");
        getch();
        exit(1);
    }
    fread(&reg, sizeof(SProducto), 1, archOrig);
    while (!feof(archOrig))
    {
        if (reg.codigo != codigoABorrar)
            fwrite(&reg, sizeof(SProducto), 1, archTemp);
        fread(&reg, sizeof(SProducto), 1, archOrig);
    }

    fclose(archOrig);
    fclose(archTemp);

    //borra el archivo original donde estaba el producto a eliminar
    remove("productos.dat");

    //hace que el archivo temporal pase a ser ahora el archivo de productos
    rename("productos.tmp", "productos.dat");
}
```

Una mejora sobre el código anterior podría ser agregar una bandera indicando si realmente se encontró o no el código a borrar y en caso de que no se haya encontrado entonces no realizar la parte de sobre escritura del archivo original.

15.3 Archivo Histórico

En algunos casos es de utilidad mantener un archivo histórico con los datos borrados por lo tanto antes de borrarlos definitivamente es posible agregar el o los registros a borrar en otro archivo para en caso de contingencia poder acceder a un dato borrado previamente este archivo se abrirá en modo append para ir siempre agregando al final los registros antes de eliminarlos definitivamente del archivo original.

16. Búsqueda en archivos binarios

Sobre el archivo pueden aplicarse los mismos algoritmos de búsqueda que sobre los vectores, pero siempre se debe tener en cuenta que el costo de recorrer un archivo completo es mayor al realizarlo en memoria.

Para separar la lógica es habitual separar la búsqueda en una función, pero en dicho caso según para que se hace la búsqueda habrá que evaluar que retornará la función de búsqueda teniendo las siguientes opciones:

- Retornar el número de registro donde se encuentra el dato buscado o un número de registro inválido sino se encuentra, por ejemplo -1. El problema de este método es que si se quiere acceder a algún dato particular

del registro deberá reposicionarse en el archivo y desplazarse hasta el número de registro indicado utilizando fseek desde el inicio del archivo

- Retornar el registro completo que contiene el dato buscado. Esta forma tiene dos posibles inconvenientes, la primera es que no se sabría el número de registro donde se encontró el dato, pero puede ser que no se lo requiera. La segunda es como detectar que no se encontró el registro buscado ya que es necesario retornar igualmente un registro (variable del tipo estructura). Una posible solución sería crear un registro con un valor especial en alguno de los campos (que no pueda existir en ningún registro) para detectar el caso de que no se encuentra.
- Retornar directamente el dato que esta estamos necesitando, generando nuevamente una convención de un dato incorrecto para detectar el caso donde no se encontró el registro. Por ejemplo, si se hace la búsqueda de un producto por su código para obtener el precio del producto entonces la función retorna directamente el precio o -1 sino lo encuentra ya que -1 no es un precio posible.

A continuación, se muestran un ejemplo de una búsqueda de producto por su código en forma secuencial. En el ejemplo desarrollado la función de búsqueda retorna una variable del tipo estructura donde sino se encuentra se guarda un -1 en el código de producto. Debido a que el programar permite realizar varias búsquedas la metodología de trabajo es mantener el archivo abierto y enviar el puntero a la función para abrir y cerrar el archivo en cada llamada.

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
typedef struct
{
    int codigo;
    char descripcion[31];
    float precioUnitario;
    char tipo;
} SProducto;

SProducto Busqueda(int, FILE *);

int main()
{
    SProducto prod;
    int aBuscar;
    FILE * ap;
    ap = fopen("productos.dat", "rb");
    if (ap==NULL)
    {
        printf("Error al abrir el archivo de productos.");
        getch();
        exit(1);
    }
}
```

```
printf("\n\nIngrese el codigo a buscar (0 fin):");
scanf("%d",&aBuscar);
while (aBuscar !=0)
{
    prod = Busqueda(aBuscar,ap);
    if (prod.codigo!=-1)
        printf("Producto %s precio es %.2f",\
            prod.descripcion,prod.precioUnitario);
    else
        printf("No existe un producto con ese codigo.");
    printf("\n\nIngrese el codigo a buscar (0 fin):");
    scanf("%d",&aBuscar);
}
fclose(ap);
return 0;
}

SProducto Busqueda(int codABuscar, FILE *ap)
{
    SProducto regProducto;
    int encontrado =0;
    rewind(ap);
    fread(&regProducto,sizeof(SProducto),1,ap);
    while(!feof(ap) && !encontrado)
    {
        if (regProducto.codigo == codABuscar)
            encontrado=1;
        else
            fread(&regProducto,sizeof(SProducto),1,ap);
    }
    if (!encontrado)
        regProducto.codigo=-1;
    return regProducto;
}
```

Nótese que al recibir el puntero del archivo abierto lo primero que se hace es volver al inicio del archivo para poder recorrerlo completo. En el ejemplo se realiza con la función `rewind` pero también puede realizarse con un `fseek` desplazándose 0 bytes desde el inicio.

17. Corte de Control

El corte de control es un proceso en el cual partiendo de registros ordenados por el valor de uno o más campos se los procesa para obtener información agrupada en lotes y sub-lotes al detectar un cambio en la/las variables por la cual la información se encuentra organizada.

En otras palabras, se procesa un conjunto ordenado de registros en subconjuntos determinados por el cambio de valor de una o más variables.

El corte de control se aplica para obtener información detallada agrupada por uno o más criterios.

Dicha información puede mostrarse en forma de reporte a medida que se detecta el cambio en la variable por la cual la información se encuentra ordenada o puede grabarse un archivo resumen con dicha información.

La condición necesaria para aplicar corte de control es que la información esté ordenada u organizada por el campo sobre el cual queremos agrupar la información, es decir que todos los registros de un mismo tipo deben estar juntos.

El corte de control puede aplicarse en varios niveles siempre y cuando la información venga ordenada por todos los campos sobre los cuales se desea agrupar. Por ejemplo, si se desea obtener información de los ingresos a un establecimiento agrupada por día y en cada día se desea obtener un detalle por hora, los datos deben estar primero ordenados por día y dentro de cada día ordenados por hora. En este caso podría aplicarse dos cortes uno por día y otro por hora a fin de obtener información agrupada en cada uno de ellos. Pero no siempre que la información esté ordenada significa que se deba aplicar corte de control. En el caso anterior si solo se desea obtener información detallada por día, entonces se hará un corte de un solo nivel ya que agrupar también por hora no aportaría información adicional.

Para aplicar corte de control si la información se ingresa por teclado debe ingresarse en forma ordenada, es por ello por lo que lo más común es procesar información ya ordenada que venga en un vector o en un archivo.

17.1 Reglas Generales para aplicar corte de control

El corte de control se realiza anidando distintas estructuras repetitivas con las siguientes reglas:

- Se realiza un ciclo repetitivo con la condición de fin de datos
- Dentro del ciclo de fin de datos se anida un ciclo por cada nivel de corte que se desea realizar (el ciclo de cada nivel va anidado dentro del ciclo del nivel anterior)
- Las condiciones de los ciclos se van propagando, es decir que el ciclo interno de corte va a incluir la condición de fin del ciclo que lo contiene y su propia condición. Si hay otros niveles de corte va a incluir las condiciones anteriores y su propia condición de fin.
- La información se lee antes de ingresar al ciclo de la condición de fin y se vuelve a leer antes de finalizar el ciclo más interno.
- Las variables que cuentan o acumulan información agrupada deben inicializar antes de comenzar el ciclo del corte y mostrarse al salir de dicho ciclo haciendo referencia a que dicha información corresponde al dato leído anteriormente y no al último, ya que el ciclo termina por que el lote finalizó y por lo tanto la lectura actual va a ser diferente.

17.2 Cómo aplicar Corte de Control en diferentes casos.

Si bien esta técnica puede aplicarse a datos ingresados por teclado, para que funcione correctamente sería necesario que los datos se carguen en orden sino daría información incorrecta o segmentada.

Por este motivo se verá directamente un ejemplo de aplicación sobre un archivo en el cual los datos fueron almacenados de forma ordenada permitiendo la aplicación de este algoritmo. Cabe aclarar que cuando se habla de datos ordenados para poder aplicar corte de control no es necesarios que los datos estén ordenados en orden ascendente o descendente por el campo sobre el cual se quiere aplicar el corte sino que simplemente se requiere que todos los registros de dicho campo vengan juntos en el archivo.

17.2.1 Un Nivel de corte desde un archivo

En la Empresa “M. y M.” se quiere saber cuánto se abona de sueldo por cada Sector de Fábrica y en toda la Empresa. Los datos están ordenados por el campo clave “Número de Sector de Fábrica”, para lo cual se han analizado los datos que se encuentra en el archivo llamado “empleados.dat” y los registros del archivo tiene el siguiente formato:

- Número de Legajo (entero)
- Apellido y Nombre (hasta un máximo de 25 letras)
- Año de Ingreso a la empresa (entero)
- Número de Sector de Fábrica (entero)
- Sueldo (real)

Para comprender el algoritmo vamos a analizar un conjunto de registros de prueba:

Legajo	Nombre	Ingreso	Sector	Sueldo
1132	Andrea Lorenzo	2001	5000	89000.00
1932	Julia Correa	2011	5000	69000.00
1423	Jorge Perez	2005	5000	79000.00
1032	Pablo Caceres	1998	7000	110000.00
1002	Julia Sarratea	1995	7000	135000.00
1133	Sandra Zarrea	2001	8000	89000.00
1145	Fernando Formento	2002	9000	49000.00
1155	Luisa Perez	2005	9000	69000.00

Como puede notarse los datos en este archivo se encuentran ordenados por Sector y lo que se busca es obtener información de cuantos empleados trabajan en cada sector.

Al estar ordenados es posible ir contando registros mientras el código de sector sea el mismo que el anterior y en momento que se lea un código de sector diferente significa que el subconjunto de datos finalizó y por lo tanto podemos mostrar la información calculada.

Legajo	Nombre	Ingreso	Sector	Sueldo
1132	Andrea Lorenzo	2001	5000	89000.00
1932	Julia Correa	2011	5000	69000.00
1423	Jorge Perez	2005	5000	79000.00
1032	Pablo Caceres	1998	7000	110000.00
1002	Julia Sarratea	1995	7000	135000.00
1133	Sandra Zarrea	2001	8000	89000.00
1145	Fernando Formento	2002	9000	49000.00
1155	Luisa Perez	2005	9000	69000.00

Se vienen leyendo
los registros uno a
uno procesando los
datos de cada uno

Al leer este registro
se detecta el cambio
de código y por lo
tanto se terminó de
procesar un sector

Volviendo al ejemplo hay 4 sectores distintos en el primer sector con código 5000 hay 3 empleados, en el segundo sector con código 7000 hay 2 empleados, luego un empleado para el sector 8000 y 2 en el 9000

Legajo	Nombre	Ingreso	Sector	Sueldo	
1132	Andrea Lorenzo	2001	5000	89000.00	
1932	Julia Correa	2011	5000	69000.00	Sector 5000
1423	Jorge Perez	2005	5000	79000.00	
1032	Pablo Caceres	1998	7000	110000.00	Sector 7000
1002	Julia Sarratea	1995	7000	135000.00	
1133	Sandra Zarrea	2001	8000	89000.00	Sector 8000
1145	Fernando Formento	2002	9000	49000.00	
1155	Luisa Perez	2005	9000	69000.00	Sector 9000

Resolución del problema: A continuación, se muestra el código que resuelve el problema con comentarios en cada sección:

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
```

```
struct PERSONA
```

```
{
    int nLegajo;
    char ApyN [26];
    int nAnio;
    int nSecFab;
    float rSueldo;
};
```

```
int main ( )
{
```

```
    struct PERSONA per;
    int antSecFab, contTotal, contSector;
    FILE *archEmpleados;
```

```
    archEmpleados = fopen("empleado.dat", "rb");
    if (archEmpleados == NULL)
```

```
    {
        printf("\nNo se puede abrir.");
        getch();
        exit(1);
    }
```

```
    contTotal = 0;
    fread(&per, sizeof(struct PERSONA), 1, archEmpleados);
    while (!feof(archEmpleados))
```

```
    {
        antSecFab = per.nSecFab;
        contSector = 0;
```

```
        do
        {
            contSector ++;
            fread(&per, sizeof(struct PERSONA), 1, archEmpleados);
        } while (per.nSecFab == antSecFab && !feof(archEmpleados));
```

```
        contTotal += contSector;
        printf("\nLa Cantidad de Empleados del Sector %d es %d", \
            antSecFab, contSector);
```

```
    }
    fclose(archEmpleados);
    printf("\nLa Cantidad de Empleados que tiene la empresa es %d", \
        contTotal);
    return 0;
}
```

Se define la estructura con el formato del registro del archivo a procesar

Se declaran las variables a utilizar y se abre el archivo en modo lectura

Antes de ciclo se inicializan los contadores y acumuladores generales (para todos los datos)

Se lee del archivo se arma el ciclo principal del proceso hasta fin de archivo

Dentro del ciclo se usan los datos leídos en este caso simplemente se cuenta y se hace la segunda lectura para pasar al siguiente registro

Se acumula en el contador general

Al salir del ciclo del corte se muestran los datos acumulados (del código anterior)

Al salir del while que indica que no hay más registros se cierra el archivo y se muestran los datos acumulados en forma general

Se guarda el código anterior y se inicializan los contadores y acumuladores parciales (por corte)

Ciclo del corte de control se repite mientras sea el mismo código que el anterior y no se llegue al final del archivo

Al ejecutar el programa con el lote de prueba planteado el resultado es el siguiente:

```
La Cantidad de Empleados del Sector 5000 es 3
La Cantidad de Empleados del Sector 7000 es 2
La Cantidad de Empleados del Sector 8000 es 1
La Cantidad de Empleados del Sector 9000 es 2
La Cantidad de Empleados que tiene la empresa es 8
```

Puede verse que se muestra un informe de la cantidad de empleados por sector, este informe se hace al salir del ciclo del corte ya que se trata de un contador parcial. Luego al finalizar el recorrido de todo el archivo se muestra el contador total de cantidad de empleados de toda la empresa.

Aclaración: el ciclo del corte de control en este ejemplo fue realizado con un ciclo do while pero también puede realizarse con un ciclo while con la salvedad que se sabe que la primera vez va a ingresar siempre. En el siguiente ejemplo el ciclo de corte fue realizado con un ciclo while.

17.2.2 Un Nivel de corte desde un archivo generando archivo resumen

Se dispone de un archivo llamado “ventas.dat” que tiene las ventas realizadas por una empresa a lo largo del mes. La empresa tiene distintas sucursales a lo largo del país. El archivo contiene registros con los siguientes datos:

- Código de sucursal (entero)
- Código de producto (entero)
- Cantidad Vendida (entero)
- Precio Unitario (real)

Para una misma sucursal se recibe un solo registro por cada producto vendido.

Se desea realizar un programa que calcule las ventas realizadas en cada sucursal generando un archivo llamado ventasXsuc.dat que contenga un registro sumariado por cada sucursal con los siguientes datos:

- Código de sucursal (entero)
- Cantidad total de productos vendidos
- Importe total recaudado
- Código de producto con mayor cantidad de unidades vendidas (considerar único)

Resolución del problema:

Para resolver este problema será necesario trabajar con los dos archivos abiertos, el archivo ventas.dat se abrirá en modo lectura para ir leyendo registro a registro y hacer el corte de control sobre el campo código de sucursal. Al mismo tiempo se debe crear el archivo ventasXsuc.dat para que cada vez que se detecte un cambio en el código de sucursal se grabé un archivo sumariado en dicho archivo.

Además de dos campos sumariados (cantidad e importe), se nos pide determinar el código de producto con mayor cantidad de unidades vendidas por lo que se deberá aplicar el algoritmo para calcular el máximo para los productos vendidos en cada sucursal.

Se deben declarar dos estructuras de datos una por cada archivo ya que almacenan registros distintos.

Estructura sVenta			
codSuc	codProd	cantVend	precioU

Estructura sResumen			
codSuc	totalVend	recaudacion	codProdMaxVtas

Nótese que este programa no tiene salida por pantalla los resultados se guardan directamente en un archivo.

Codificación en Lenguaje C:

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

struct sVenta
{
    int codSuc;
    int codProd;
    int cantVend;
    float precioU;
};
struct sResumen
{
    int codSuc;
    int totalVend;
    float recaudacion;
    int codProdMaxVtas;
};
int main()
{
    FILE * arch_Vta, * arch_VtaXSuc;
    struct sVenta vta;
    struct sResumen resumen;
    int primero, max;
    arch_Vta = fopen("ventas.dat", "rb");
    arch_VtaXSuc = fopen("ventasxsuc.dat", "wb");
    if (arch_Vta==NULL || arch_VtaXSuc==NULL)
    {
        printf("Error al abrir los archivos");
        getch();
        exit(1);
    }
    fread(&vta, sizeof(vta), 1, arch_Vta);
    while (!feof(arch_Vta))
    {
        /*guarda la sucursal anterior e inicializa los acumuladores y contadores directamente en
        la variable del tipo estructura que se va a grabar el archivo por sucursal*/
        resumen.codSuc = vta.codSuc;
        resumen.totalVend=0;
        resumen.recaudacion =0;
        primero = 1; //para el maximo

        while (!feof(arch_Vta) && resumen.codSuc == vta.codSuc)
        {
            resumen.totalVend += vta.cantVend;
            resumen.recaudacion += vta.cantVend * vta.precioU;

            if(primerero || vta.cantVend > max)
            {
                max = vta.cantVend;
                resumen.codProdMaxVtas = vta.codProd;
                primero = 0;
            }
            fread(&vta, sizeof(vta), 1, arch_Vta);
        }
        fwrite (&resumen, sizeof( resumen),1, arch_VtaXSuc);
    }
    fclose(arch_Vta);
    fclose(arch_VtaXSuc);
}
```

17.2.3 Un Nivel de corte desde un archivo generando archivos dinámicamente

El sistema de control de ingreso y salida de empleados de una fábrica deja al final del mes un archivo ordenado por sector y sumariado por empleado llamado horasTrabajadas.dat con los siguientes datos:

- Sector (texto de 10 caracteres máximo)
- DNI del empleado (entero)
- Horas trabajadas

Por otro lado se dispone del archivo empleados.dat con los empleados de la fábrica que contiene los siguientes datos:

- DNI del empleado (entero)
- Nombre y Apellido (texto de 30 caracteres máximo)
- Valor que cobra por hora (float)

No se sabe la cantidad exacta de empleados pero sí se sabe que no son más de 100.

Se desea realizar un programa que utilizando ambos archivos genere para cada sector un archivo que contenga el detalle de sueldos que deba pagar a sus empleados. Se debe generar para cada sector un archivo distinto cuyo nombre sea nombreSector.dat y debe contener registros con la siguiente información:

- DNI del empleado (entero)
- Nombre y Apellido (texto de 30 caracteres máximo)
- Horas trabajadas (entero)
- Sueldo a Pagar (float)

Resolución del problema:

Inicialmente se deben descargar en memoria los datos los empleados de la empresa, que como se sabe que no son más de 100 se pueden descargar en un vector.

Luego se procesa el archivo horasTrabajadas.dat que está ordenado por sector. Por cada sector se debe generar dinámicamente un nuevo archivo con el nombre dicho sector y calcular en él la información solicitada para ello se debe buscar el empleado en el vector en memoria para obtener su nombre y valor que cobrar por hora.

Se necesitan tres estructuras una por cada archivo (en realidad para los sectores se crean varios archivos, pero todos con la misma estructura):

Estructura est_horas			
sector	dni	horas	

Estructura			
dni	AyN	valorHora	

Estructura est_sector			
dni	AyN	totalHoras	sueldo

Codificación en Lenguaje C:

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <string.h>
```

```
typedef struct
{
    char sector[11];
    int dni;
    int horas;
} est_horas;

typedef struct
{
    int dni;
    char AyN[31];
    float valorHora;
} est_empleado;

typedef struct
{
    int dni;
    char AyN[31];
    int totalHoras;
    float sueldo;
} est_sector;

int CargaEmpleados (est_empleado[],int);
int BuscarEmpleado (est_empleado[],int,int);

int main()
{
    FILE * pHoras, * pSector;
    int cantEmpleados, pos;
    est_empleado vEmp[100];
    est_horas regHora;
    est_sector regSector;
    char secAnt[11], nombreArchivoSector[15];

    cantEmpleados = CargaEmpleados(vEmp, 100);
    pHoras = fopen("horasTrabajadas.dat", "rb");
    if (pHoras == NULL)
    {
        printf ("No se pudo abrir el archivo de horas trabajadas");
        getch();
        exit (1);
    }
    fread(&regHora, sizeof(regHora), 1, pHoras);
    while (!feof(pHoras))
    {
        strcpy(secAnt, regHora.sector);
        /*Se genera el nombre del archivo basado en el nombre del sector */
        strcpy (nombreArchivoSector, regHora.sector);
        strcat(nombreArchivoSector, ".dat");
        pSector = fopen (nombreArchivoSector, "wb");
        if (pSector ==NULL)
        {
            printf ("No se pudo crear el archivo para el sector %s", secAnt);
            getch();
            exit(1);
        }

        while (!feof(pHoras) && strcmp(secAnt, regHora.sector)==0)
        {
            /*se busca el empleado y como se trabaja con archivos ya creados
            se asume que siempre lo encuentra*/
            pos = BuscarEmpleado(vEmp,regHora.dni, cantEmpleados);
            regSector.dni = regHora.dni;
            strcpy(regSector.AyN, vEmp[pos].AyN);
            regSector.totalHoras = regHora.horas;
            regSector.sueldo = regHora.horas * vEmp[pos].valorHora;
            fwrite(&regSector, sizeof(regSector),1 , pSector);

            fread(&regHora, sizeof(regHora), 1, pHoras);
        }
    }
}
```

```

        fclose(pSector);
    }
    fclose(pHoras);
}
int CargaEmpleados (est_empleado ve[],int max)
{
    FILE *pEmpleados;
    int i=0;
    est_empleado regEmpleado;
    pEmpleados = fopen("empleados.dat", "rb");
    if (pEmpleados == NULL)
    {
        printf ("No se pudo abrir el archivo de empleados.");
        getch();
        exit (1);
    }

    fread(&regEmpleado, sizeof(regEmpleado),1, pEmpleados);
    while (!feof(pEmpleados) && i < max)
    {
        ve[i] = regEmpleado;
        i++;
        fread(&regEmpleado, sizeof(regEmpleado),1, pEmpleados);
    }
    fclose(pEmpleados);
    return i;
}
int BuscarEmpleado (est_empleado ve[],int dni,int tam)
{
    int i =0 , pos =-1;
    while (pos== -1 && i<tam)
    {
        if (ve[i].dni == dni)
            pos = i;
        else
            i++;
    }
    return pos;
}

```

17.2.4 Dos Niveles de corte desde un archivo

El corte de control puede aplicarse en distintos niveles si es necesario, pero para ello cada campo por el cual se quiera aplicar el corte debe estar ordenado. En el caso de un corte de dos niveles los datos vendrán ordenados por el campo por el cual se hará el primer nivel de corte y luego dentro de ese subconjunto de datos vendrán ordenados por un segundo campo. Se recuerda que cada nivel de corte agregará un ciclo para esperar que cambien los datos anidados dentro del ciclo anterior y propagando la condición de los anteriores. La segunda lectura del archivo se debe realizar el ciclo más interno.

A modo de ejemplo se presenta el siguiente problema:

Se dispone de un archivo binario llamado inscripciones.dat que contiene la información de los inscriptos a la universidad. El formato de registro es el siguiente:

- Departamento (texto de 15 caracteres máximo)
- Código de materia (entero)
- Dni del alumno inscripto

Los datos en este archivo se encuentran ordenados por departamento y dentro de cada departamento por código de materia.

Realizar un programa que indique la cantidad de inscriptos a cada materia, la cantidad de inscriptos a cada departamento y la cantidad de inscriptos en total a toda la universidad. Además indicar la cantidad de materias por departamento que tuvieron inscripciones.

Resolución:

Para realizar el análisis se plantea el siguiente archivo de pruebas donde puede verse que los datos están organizados por departamento y luego dentro de cada departamento por materia. Nótese que incluso los códigos de materia pueden repetirse en entre departamentos, pero se tratan de materias distintas y el proceso realizado debe poder diferenciarlas. En los datos planteados la materia con código 3800 se encuentra tanto en ingeniería como en Salud.

Departamento	Materia	DNI
Ingenieria	3623	42122148
Ingenieria	3623	41822145
Ingenieria	3623	36172178
Ingenieria	3623	48122142
Ingenieria	3732	12122147
Ingenieria	3732	32122126
Ingenieria	3800	41122147
Salud	3800	26221148
Salud	3800	26124147
Salud	3800	22122142
Salud	3900	12541141
Salud	3900	32172145
Salud	3900	42444144
Derecho	1200	40122148
Derecho	1200	40145785
Derecho	1580	36212118
Derecho	1580	33781578
Derecho	3000	34841589
Derecho	3000	41578488

Corte por materia

Corte por materia

Corte por departamento

Corte por departamento

Al tener dos niveles de corte hay que analizar cada uno de los informes solicitados y determinar si se tratan de información a nivel general (para todo el archivo), por departamento (sobre el primer nivel de corte) o por materia (segundo nivel de corte). De ello dependerá donde se inicializaran los contadores, donde se incrementan y donde se muestran.

Los informes solicitados son:

- la cantidad de inscriptos a cada materia: para este cálculo se requiere el segundo nivel de corte

- la cantidad de inscriptos a cada departamento: para este cálculo se requiere el primer nivel de corte
- la cantidad de inscriptos en total a toda la universidad: este es un contador general para todos los datos del archivo
- la cantidad de materias por departamento que tuvieron inscripciones: este calculo si bien es por departamento (primer nivel de corte) necesita saber la cantidad de materias diferentes y por lo tanto aprovechará el segundo nivel de corte para no contar materias repetidas.

Código:

```
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct
{
    char dpto[16];
    int materia;
    int dni;
}inscripcion;

int main()
{
    FILE * archInsc;
    char dptoAnterior[16];
    int materiaAnterior, total, totalDpto, totalMateria, cantMaterias;
    inscripcion inscripto;
    archInsc = fopen("inscriptos.dat","rb");
    if (archInsc==NULL)
    {
        printf("Error en el archivo inscriptos.");
        getch();
        exit(1);
    }

    total =0; //inicializadon del contador general
    fread(&inscripto, sizeof(inscripcion),1, archInsc);
    while (!feof(archInsc))
    {
        printf("----- Departamento %s-----\n",inscripto.dpto);
        strcpy (dptoAnterior, inscripto.dpto);
        totalDpto=0; //contadores por departamento
        cantMaterias=0;
        do //ciclo del corte por departamento
        {
            materiaAnterior = inscripto.materia;
            totalMateria=0; //contador por materia

            do //ciclo del corte por materia
            {
                totalMateria++;
                fread(&inscripto, sizeof(inscripcion),1, archInsc);
            }while(!feof(archInsc) \
                && strcmpi(dptoAnterior, inscripto.dpto)==0 \
                && materiaAnterior == inscripto.materia);

            printf("\nMateria %d Inscriptos %d",materiaAnterior, totalMateria);
            totalDpto+=totalMateria;
            cantMaterias++;
        }while(!feof(archInsc) && strcmpi(dptoAnterior, inscripto.dpto)==0);
    }
}
```

```

    printf("\n\nTotal Inscriptos al departamento: %d", totalDpto);
    printf("\nCantidad de materias: %d\n\n", cantMaterias);
    total +=totalDpto;
}
printf("-----\n");
printf("Total Inscriptos a la Universidad: %d\n", total);

fclose(archInsc);
return 0;
}

```

Resultado de la ejecución:

```

----- Departamento Ingenieria-----

Materia 3623 Inscriptos 4
Materia 3732 Inscriptos 2
Materia 3800 Inscriptos 1

Total Inscriptos al departamento: 7
Cantidad de materias: 3

----- Departamento Salud-----

Materia 3800 Inscriptos 3
Materia 3900 Inscriptos 3

Total Inscriptos al departamento: 6
Cantidad de materias: 2

----- Departamento Derecho-----

Materia 1200 Inscriptos 2
Materia 1580 Inscriptos 2
Materia 3000 Inscriptos 2

Total Inscriptos al departamento: 6
Cantidad de materias: 3

-----
Total Inscriptos a la Universidad: 19

```